

Developing a Firewall to protect users data

(Try smarter)

Abstract

This project (Try Smarter), presents an intelligent system developed to enhance the performance of built-in firewalls in operating systems, enabling them to detect and block port scanning patterns dynamically

The project was implemented using **Python** and **Scapy** library.

The system monitors incoming TCP traffic in real time and identifies suspicious packets that indicate reconnaissance and scanning activities, then dynamically adds blocking rules to the built-in firewall.

Several tests were conducted, demonstrating the project's effectiveness, The attacker's scanning time increased from **0.55** seconds to more than **21** seconds, while the system successfully blocked over **4000** malicious packets

This solution does not require any additional hardware or software, making it a simple, effective, and cost efficient option for standard systems.

1.Introduction, Aims and Objectives

1.1 Introduction

The world of cybersecurity is rapidly evolving, and the threats within this domain are advancing at a very fast pace. As a result, it has become essential to take awareness and protection seriously, whether through costly commercial solutions or by enhancing the tools integrated into operating systems. The latter is the approach adopted in this project because it is both effective and cost efficient. Most users depend on the built-in firewalls of operating systems as their first line of defense, however these firewalls significantly lack the ability to detect suspicious patterns and respond to them dynamically.

This report presents the **Try Smarter** project. In this project we developed a simple intelligent system designed to enhance the capabilities of built-in operating system firewalls.

Try Smarter strengthens firewalls by adding dynamic detection and blocking features for reconnaissance activities that may originate from external devices with the purpose of gathering information for a larger attack.

1.2 Background

The reconnaissance phase is considered the starting point of any cyberattack, and in practice, this step largely determines the outcome of the attack.

The success of any attack depends heavily on the information gathered during reconnaissance.

One of the most prominent tools used in this phase is undoubtedly **Nmap**.

This tool helps attackers obtain:

- Open ports
- Running services
- Operating system versions

The fundamental problem is that built-in system firewalls cannot detect activities such as Nmap scanning, giving attackers an excellent opportunity to collect information.

This challenge inspired the idea behind this project to address that security gap.

1.3 Aims

As mentioned above, the goal is to develop a simple and intelligent system that enhances the performance of built-in firewalls so that they can:

- Detect port-scanning activities.
- Perform dynamic blocking.
- Mock the attacker before blocking them by sending the message: **Try smarter**.

1.4 Objectives

To achieve the previous aims, the following objectives must be accomplished:

- .Gain a deep understanding of how TCP SYN scans work and how they are used for reconnaissance.
- .Design a detection logic capable of distinguishing between normal and suspicious network traffic.
- .Develop a **Python** script to monitor network traffic.
- .Integrate the script with **iptables** to enable the addition of blocking rules.
- .Set up a testing environment using **VMware**.
- .Document the results and compare the level of security before and after implementation.

1.5 Report Structure

This report is divided into seven main chapters as follows:

(Research):

A review of the literature and scientific resources related to port scanning and defense mechanisms against it.

(Analysis):

This chapter will be integrated with the research section and will provide a detailed analysis of the problem using diagrams to understand the system requirements.

(Design):

This chapter explains the architectural design and the algorithm logic.

(Implementation):

This chapter describes the coding process, system installation, and the technical challenges encountered.

(Testing and Results):

This chapter presents the project results and includes a comparison of the security level before and after implementation.

(Conclusion and Future Work):

This chapter evaluates the extent to which the project objectives were achieved and proposes future improvements for the system.

2.research and Analysis

2.1

In our current digital age, computers have become an integral part of our daily lives, whether in educational, professional, or entertainment environments.

Users store enormous amounts of sensitive data on their devices, including personal documents, credentials, and various files. Once a device is connected to the internet, it becomes a potential target for malicious actors seeking to exploit vulnerabilities and steal data.

This project aims to develop an intelligent rule set for the operating system's built-in firewall. These rules will monitor, detect, and dynamically block **reconnaissance** attempts without the need for costly commercial solutions.

The term **reconnaissance** was mentioned above, but what does reconnaissance mean, and why is it dangerous?

In the field of cybersecurity, reconnaissance is considered the first and most critical stage of any successful cyberattack according to the **Intrusion Kill Chain** model developed by Hutchins and colleagues in 2011.

This model explains that disrupting any stage of an attack leads to stopping the entire attack. Looking at our project, it specifically targets the reconnaissance phase. Therefore, the rules we are developing will effectively terminate the attack process at its very beginning, meaning the attacker will be unable to gather any information required to continue the attack.

When discussing reconnaissance, it is important to mention that one of the most significant tools used during this phase is **Nmap**.

Nmap(network mapper) is an open-source tool used for network discovery and security auditing.

This tool relies on IP packets to collect information about a target, such as port status, service names and versions, operating system type, and device MAC addresses

2.2

How does port scanning work[5] ?

Nmap uses what is known as a **TCP SYN** scan.

This type of scan is extremely fast and highly stealthy, capable of scanning thousands of ports per second.

Lyon explains in his 2009 book **Nmap Network Scanning** that this type of scanning is difficult for traditional firewalls to detect, making the development of defensive mechanisms essential.

This scan is based on the **TCP Three-Way Handshake**, a process used by the TCP protocol to establish a reliable connection between two devices. It consists of three stages[4]:

- The client sends a **SYN** packet to the target port.
- If the port is open, it responds with a **SYN-ACK** packet.
- The client completes the connection by sending an **ACK** packet.

In port scanning, the attacker sends multiple SYN packets to different ports and monitors the responses to determine which ports are open.

.Why do I want to improve the built-in firewall instead of purchasing a commercial solution?

A 2022 study by Brown, published by the SANS Institute, indicates that enhancing the configuration of an existing firewall is a strategic, intelligent, and cost-effective defensive solution, particularly for small and medium-sized systems that do not require the advanced features of commercial firewalls.

This is exactly what we aim to achieve through this project: enhancing the performance of the operating system's built-in firewall to provide a higher level of protection without additional costs.

2.3 Literature Review

This research, in the section on the Intrusion Kill Chain, discusses the stages through which a full intrusion process takes place. It explains that disrupting any one of these stages leads to halting the entire process. The first stage here is reconnaissance, and as we explained earlier, the rule we are developing addresses the attacker during the reconnaissance phase. In doing so, we effectively terminate the attack completely before it even begins. This directly informed our decision to focus the detection logic on SYN packet patterns, as intercepting reconnaissance activity prevents any further stages of the attack from occurring.[1]

Port scanning is considered the first and most critical step in any cyberattack. In this book, the author discussed the TCP SYN scan, stating that this type of scan can probe thousands of ports within a single second. This process is characterized by speed and stealth, making it difficult for traditional firewalls to detect. This finding justified our decision to implement the detection mechanism using the Scapy library at the kernel level, ensuring real-time packet interception rather than relying on slower application-level monitoring.[2]

This research also highlighted that improving the configuration of an existing firewall is a smart and cost-effective defensive strategy, rather than purchasing devices such as Next-Generation Firewalls, most of whose services may not be necessary in smaller systems. Here, we can clearly see the economic importance of our project.[3]

While Lyon highlights the speed and stealth of port scanning attacks, Brown emphasizes the value of cost-effective defensive solutions. Together, these two perspectives form the foundation of our project design: a system that responds to fast, stealthy attacks using only the built-in tools available in standard operating systems, with no additional cost required.

2.4 Analysis

In this section, I will attach several images that illustrate the problem we are working on and the solution that we will implement from all aspects.

Figure1

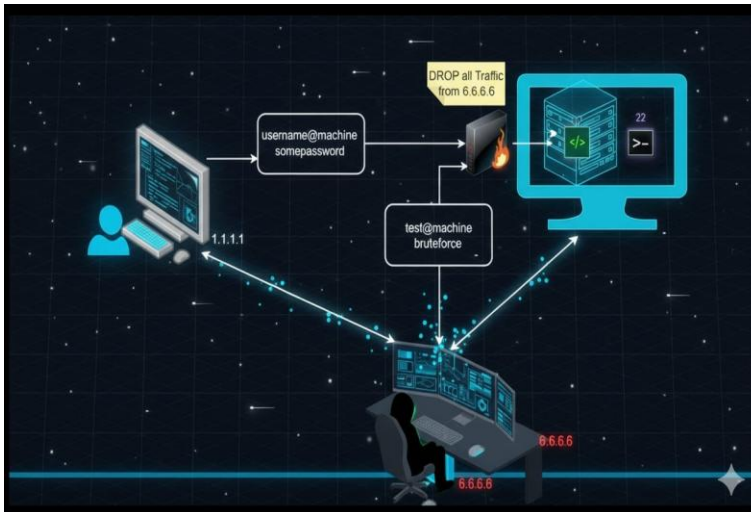


Figure1 illustrates two individuals: one holds the IP (1.1.1.1) and is trusted, while the other holds the IP (6.6.6.6) and is the attacker. As we can see, there is a rule on the firewall that states all traffic from the IP (6.6.6.6) must be dropped. In this way, the attacker will be blocked as soon as they attempt to connect. The firewall will check the IP, compare it with the existing rule, and act accordingly. Thus, no connection will be established between the attacker and the target machine

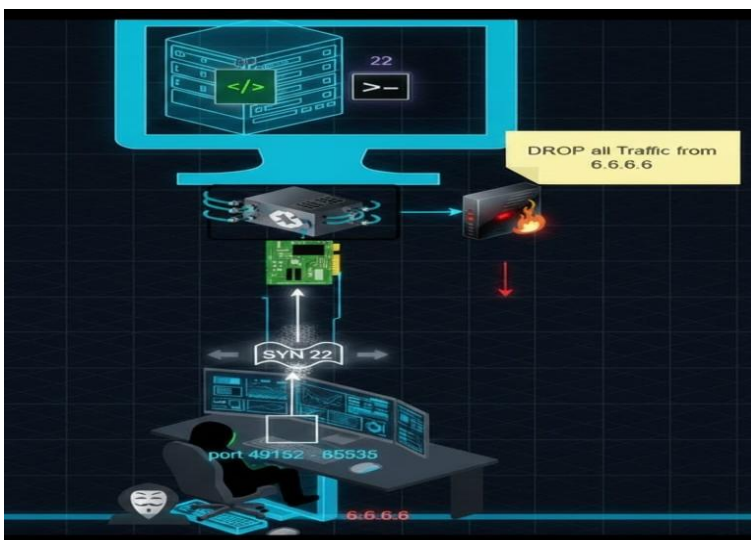
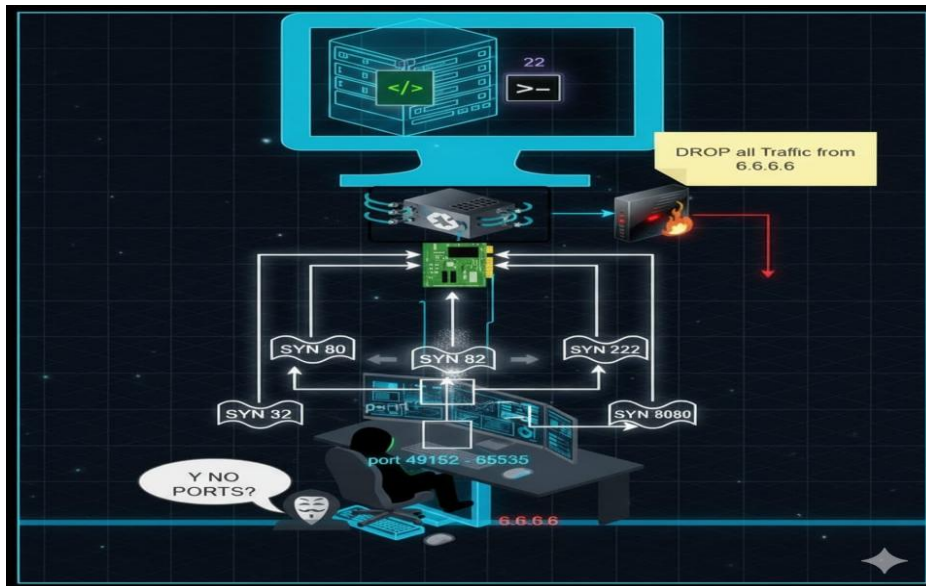


Figure2 , we explain the attacker's mechanism from the perspective of the three-way handshake. The attacker sends a SYN flag to the server, which is received by the network and passed to the kernel. At this point, the IP is verified, and the attacker will not receive a SYN/ACK response, and no connection will be established between the attacker and the target machine.

Figure3



In figure3 , the attacker attempts to reach the target machine using the tool nmap. They send multiple SYN flags to different ports. However, in the same way as before, the attacker will be blocked by the firewall rule. For this reason, the attacker will not be able to determine which ports are open and which are closed.

The most important question now is: how did we know that we should block the IP(6.6.6.6)?! In other words, how did we identify the attacker's IP?

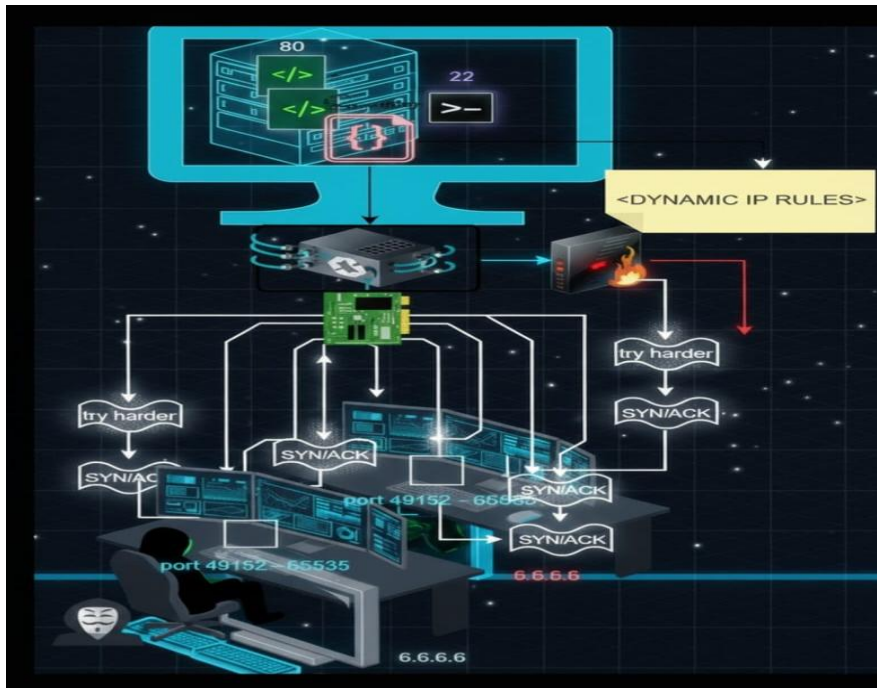
Simply put, this is the core of our project. We will define the malicious actor as the one who performs suspicious activity. In the context of our project, suspicious activity is sending multiple packets to different ports on the target machine—essentially using nmap to scan the machine's ports and services. This is considered suspicious behavior.

From this, we can derive a rule to block this category of attackers. This means we achieve a certain level of protection, as the attacker will not get another chance and their IP will be recorded on the firewall.

As for how to install the rule and make this process dynamic on the firewall, that will be discussed in the next section.

After that, the scapy library will identify any entity that sends multiple SYN flags to different ports as a malicious actor. It will then add the attacker's IP address to the firewall's block list and exclude all of their packets, and no connection will be established between the attacker and the target machine.

Figure5



For a bit of fun, before blocking the attacker, we will add some extra details to Scapy. First, it will send the attacker a SYN/ACK response, just as they expect. Immediately afterward, it will send them the phrase “try smarter” with different packets.

After this, the attacker's IP address will be added to the firewall's block list, ensuring that they will never be able to reach the target machine again.

This section has presented a visual and analytical overview of the problem and proposed solution. The diagrams illustrate the complete attack scenario, from the initial SYN flood to the dynamic blocking mechanism, The system requirements have been clearly defined to guide the design and implementation phases that follow

3.Design

In this chapter, we will discuss the design decisions I made while building **Try Smarter**. These design decisions include both the system architecture design and the algorithm design, along with the rationale behind each decision.

3.1 System Architecture

In **Try Smarter**, I adopted a simple architecture consisting of three layers:

- Monitoring Layer
- Detection Layer
- Response Layer

Why did I choose this design?

Simply put, this architecture ensures a clear separation of responsibilities, making the system easier to extend and modify in the future.

3.2 Algorithm Design

3.2.1 Detection Logic

Input:Incoming TCP packet

Output:Block or allow decision

. Does the packet contain IP and TCP layers?

- No → Ignore , Yes → Continue

.Is the flag equal to SYN only?

- No → Ignore , Yes → Continue

. Is the IP already blocked?

- Yes → Ignore , No → Continue

. Increment the SYN counter for this IP by **1**

.Is the counter greater than 5?

- Yes → Send **Try Smarter** and then block the IP
- No → Send SYN-ACK

3.2.2

We also added logic to remove blocks every 10 minutes:

Every 5 seconds:

- Check the list of blocked IPs
- Have 10 minutes passed since the block was applied?
- No → Continue
- Yes → Remove the rule from iptables

3.3 System Component Design

3.3.1 Programming Language Selection

I chose Python for several reasons, including:

- Support for the Scapy library
- Easy integration with operating system commands through `subprocess`
- Suitable for building professional security tools
- Large support community

It would have been possible to use C/C++, but they are more difficult in terms of development and maintenance. Another option was Bash, but it is not suitable for packet handling.

3.3.2 Choosing the Scapy Library

The first and most important reason is that it works directly at the kernel level.

Other reasons include:

- Packet capture and analysis capabilities
- Support for network protocols
- Easy installation on Linux

3.3.3 Choosing iptables

- Built into Linux
- Supports dynamic addition and removal of rules
- Also operates at the kernel level

4.3.4 Threshold Design

Before blocking, we set a threshold of 5 SYN packets as the maximum limit. This number was chosen based on the following considerations:

- Normal connections typically do not generate more than 3–4 simultaneous SYN connections
- Nmap sends hundreds of SYN packets per second
- It is a reasonable value for reducing false positives

3.3.5 Threading Design

We used a two-thread model to run two parallel processes:

- Packet monitoring process
- Process for checking IPs whose blocking period has expired

This ensures continuous monitoring without interruption.

3.4 Test Environment Design

component	details	justification
Vmware Workstation	Virtual environment	Ecellent support to the isolated networks
Ubuntu 64 bit	Victim device	Linux environment support scapy and iptables
Kali linux	Attacker device	Includes Nmap
Nat network	Type of network	Isolate the environment while allowing devices to communicate

3.5 Summary

The design of **Try Smarter** was based on the principles of simplicity, effectiveness, and future scalability.

Every design decision I made was supported by clear justifications that contributed to achieving the objectives of my project.

4. Implementation:

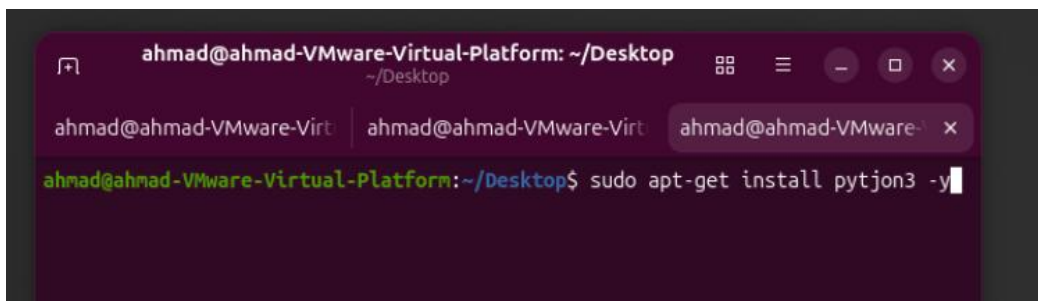
In this section, we will review the implementation process of the Try Smarter system from start to finish. We will write the code and discuss the technical challenges we encountered and how we overcame them.

4.1 System Requirements:

component	Requirement
OS	Linux(ubuntu)
python	3.8
scapy	2.4
privilege	Sudo\root
Ram	512MB at lleast

4.2 Installation Steps

- Install Python



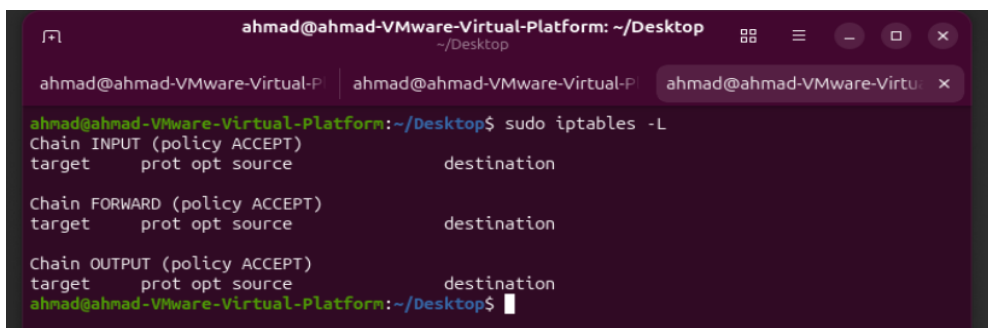
```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$ sudo apt-get install python3 -y
```

-install Scapy



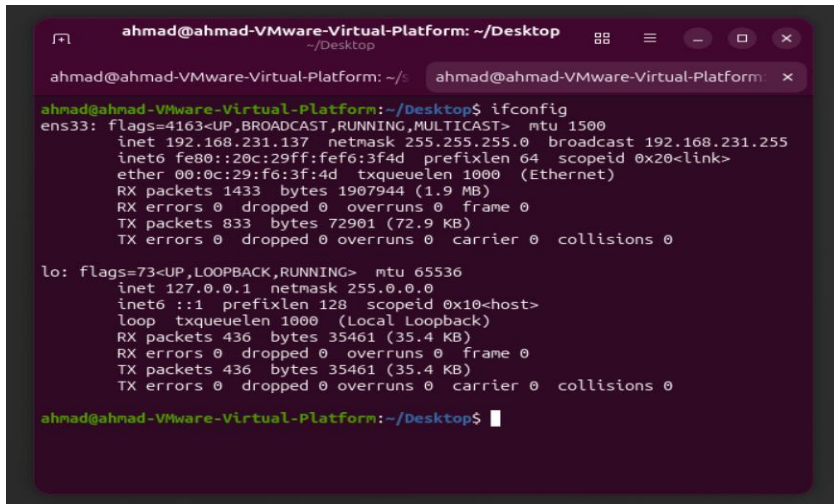
```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$ sudo apt-get install python3-scapy -y
```

- Verify iptables



```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target prot opt source destination
Chain FORWARD (policy ACCEPT)
target prot opt source destination
Chain OUTPUT (policy ACCEPT)
target prot opt source destination
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$
```


- Identify the network interface

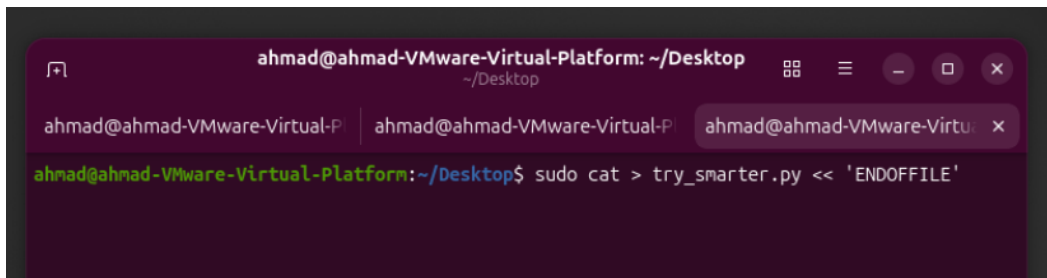


```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.231.137 netmask 255.255.255.0 broadcast 192.168.231.255
    inet6 fe80::20c:29ff:fef6:3f4d prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:f6:3f:4d txqueuelen 1000 (Ethernet)
    RX packets 1433 bytes 1907944 (1.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 833 bytes 72901 (72.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 436 bytes 35461 (35.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 436 bytes 35461 (35.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

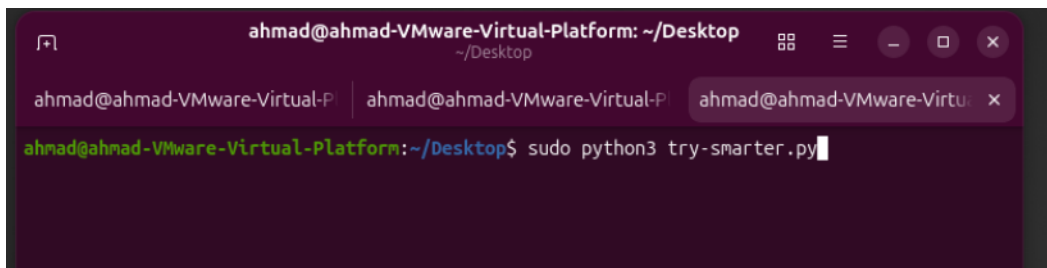
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop$
```

- Create a file named Trysmarter.py to Place the code inside it:



```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop$ sudo cat > try_smarter.py << 'ENDOFFILE'
```

- Run the script:



```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop$ sudo python3 try-smarter.py
```

4.3 Code and Explanation:

4.3.1 Imports and Basic Configuration:

```
1 from scapy.all import *
2 import subprocess
3 from collections import defaultdict
4 from datetime import datetime, timedelta
5 import time
6 import threading
7
8 # إعدادات النظام
9 scan_tracker = defaultdict(lambda: {"count": 0, "timestamp": None})
10 BLOCK_DURATION = timedelta(minutes=10)
11 INTERFACE = "ens33"
12 unblock_tasks = []
```

This code includes:

- A Scan Tracker to track IP packets
- A Block Duration of 10 minutes
- The monitored network interface
- Unblock Tasks for removing blocks

4.3.2 block and unblock:

```
def block_ip(ip): 1 usage
    if is_ip_blocked(ip):
        print(f"[!] IP {ip} is already blocked.")
        return
    subprocess.run(["sudo", "iptables", "-I",
                    "INPUT", "1", "-s", ip, "-j", "DROP"])
    print(f"[OK] IP {ip} successfully blocked.")

def unblock_ip(ip):
    subprocess.run(["sudo", "iptables", "-D",
                    "INPUT", "-s", ip, "-j", "DROP"])
    print(f"[OK] IP {ip} successfully unblocked.")
```

The code here adds the rule to the top of the list to ensure it is applied immediately.

It also removes the rule when the blocking period expires.

4.3.3 The function responsible for packet processing:

```
26  def handle_packet(packet):
27      if not (IP in packet and TCP in packet):
28          return
29      if packet[TCP].flags == "S":
30          src_ip = packet[IP].src
31          if src_ip == "127.0.0.1":
32              return
33          if is_ip_blocked(src_ip):
34              return
35
36          scan_tracker[src_ip]["count"] += 1
37
38          if scan_tracker[src_ip]["count"] > 5:
39              # رسالة Try Smarter! ثم الحجب
40              send_try_smarter(packet)
41              block_ip(src_ip)
42          else:
43              # رسالة تحذير مبكر
44              send_try_smarter(packet)
```

4.3.4 The function responsible for sending the "Try Smarter" message.

```
45  def send_try_smarter(packet): 2 usages
46      src_ip = packet[IP].src
47      port = packet[TCP].dport
48      src_port = packet[TCP].sport
49
50      syn_ack = (
51          IP(dst=src_ip, src=packet[IP].dst) /
52          TCP(sport=port, dport=src_port,
53              flags="SA", seq=100,
54              ack=packet[TCP].seq + 1)
55      )
56      send(syn_ack, verbose=0, iface=INTERFACE)
57
58      data_packet = (
59          IP(dst=src_ip, src=packet[IP].dst) /
60          TCP(sport=port, dport=src_port,
61              flags="PA", seq=101,
62              ack=packet[TCP].seq + 1) /
63          Raw(load="Try Smarter!")
64      )
65      send(data_packet, verbose=0, iface=INTERFACE)s
```

4.4 Challenges I faced:

challenge	Cause	Solution
Iptables rule didn't work	Rule was added at the end of the chain using -A, so it was processed after existing ACCEPT rules	Replaced -A with -I INPUT 1 to insert the rule at the top of the chain
Scapy not installed	Not included by default in Ubuntu	Installed using sudo apt-get install python3-scapy
name_ error	Copy-paste issue through nano caused special characters to be misread	Replaced nano with cat > command to write the file directly
AttributeError: unblock_tasks	Class attribute referenced before the object was properly initialised	Moved unblock_tasks to a global variable instead of a class attribute
Try Smarter message not reaching Kali	Packets were being sent as broadcast due to missing interface specification	Added iface=INTERFACE parameter to all send () calls
Conflict between UFW, Snort and iptables	Multiple security tools controlling the same firewall rules simultaneously	Temporarily disabled UFW and Snort during testing to isolate iptables behaviour

4.5

The Try Smarter system was successfully implemented after overcoming numerous technical challenges. Every challenge we faced served as a valuable learning opportunity. The final version of the code now works excellently and fulfills all the required functionalities.

5. Testing and Results

In this chapter, we will conduct multiple tests on the Try Smarter system. The purpose of these tests is to verify the system's effectiveness in detection and automatic blocking.

In this section, I will test the tool at three main levels:

First: Without any protection

Second: With an iptables rule configured manually

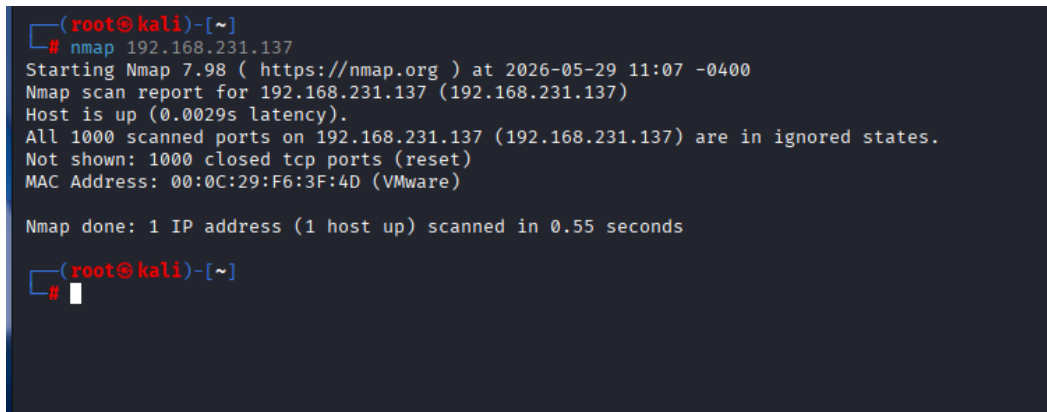
Third: Dynamically using the script

5.1 Test Without Protection

The objective here is to test the current security state.

Procedures performed:

- Running tcpdump on Ubuntu to monitor incoming network traffic.
- Running nmap from Kali against Ubuntu.



```
(root@kali)-[~]
# nmap 192.168.231.137
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 11:07 -0400
Nmap scan report for 192.168.231.137 (192.168.231.137)
Host is up (0.0029s latency).
All 1000 scanned ports on 192.168.231.137 (192.168.231.137) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 00:0C:29:F6:3F:4D (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.55 seconds

(root@kali)-[~]
#
```

- (Figure 1) The system performs monitoring without any additional steps

```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop...
~/Desktop
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop$ sudo tcpdump -i ens33 'tcp[flags] & tcp-syn !=0'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:41:04.516906 IP 192.168.231.129.52625 > 192.168.231.137.http-alt: Flags [S],
seq 627662641, win 1024, options [mss 1460], length 0
18:41:04.516911 IP 192.168.231.129.52625 > 192.168.231.137.ssh: Flags [S], seq 6
27662641, win 1024, options [mss 1460], length 0
18:41:04.516912 IP 192.168.231.129.52625 > 192.168.231.137.sunrpc: Flags [S], se
q 627662641, win 1024, options [mss 1460], length 0
18:41:04.516912 IP 192.168.231.129.52625 > 192.168.231.137.http: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
18:41:04.516913 IP 192.168.231.129.52625 > 192.168.231.137.epmap: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
18:41:04.516913 IP 192.168.231.129.52625 > 192.168.231.137.auth: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
18:41:04.516914 IP 192.168.231.129.52625 > 192.168.231.137.domain: Flags [S], se
q 627662641, win 1024, options [mss 1460], length 0
18:41:04.517289 IP 192.168.231.129.52625 > 192.168.231.137.1720: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
18:41:04.517290 IP 192.168.231.129.52625 > 192.168.231.137.pop3: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
18:41:04.517291 IP 192.168.231.129.52625 > 192.168.231.137.imap2: Flags [S], seq
627662641, win 1024, options [mss 1460], length 0
```

- (Figure 2) Scan result without protection

- Results Table:

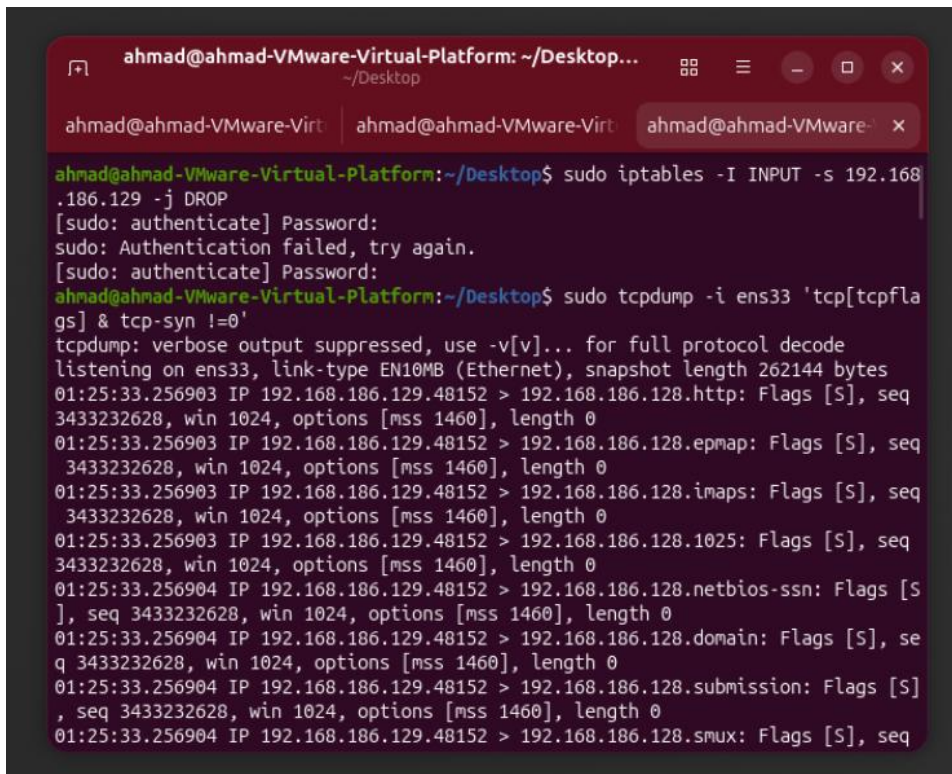
port	Closed
Time to scan	0.55 sec
Info kali get	Know that ubuntu available
Blocked packets	00

Analysis:

The result **closed** means that the device responds with an RST packet for every SYN packet, which informs the attacker that the host exists and is active on the network.

This means that the attacker is able to complete the reconnaissance process.

5.2 Manual Blocking Test



A terminal window titled 'ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop...' shows the execution of two commands. First, 'sudo iptables -I INPUT -s 192.168.186.129 -j DROP' is entered, followed by a password prompt and a failed authentication message. Then, 'sudo tcpdump -i ens33 'tcp[tcplflags] & tcp-syn !=0'' is entered. The output shows tcpdump listening on ens33 and capturing several SYN packets from 192.168.186.129 to various ports on 192.168.186.128, including http, epmap, inmap, 1025, netbios-ssn, domain, submission, and smux.

```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop...  
ahmad@ahmad-VMware-Virt | ahmad@ahmad-VMware-Virt | ahmad@ahmad-VMware-Virt x  
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$ sudo iptables -I INPUT -s 192.168.186.129 -j DROP  
[sudo: authenticate] Password:  
sudo: Authentication failed, try again.  
[sudo: authenticate] Password:  
ahmad@ahmad-VMware-Virtual-Platform:~/Desktop$ sudo tcpdump -i ens33 'tcp[tcplflags] & tcp-syn !=0'  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
01:25:33.256903 IP 192.168.186.129.48152 > 192.168.186.128.http: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256903 IP 192.168.186.129.48152 > 192.168.186.128.epmap: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256903 IP 192.168.186.129.48152 > 192.168.186.128.inmap: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256903 IP 192.168.186.129.48152 > 192.168.186.128.1025: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256904 IP 192.168.186.129.48152 > 192.168.186.128.netbios-ssn: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256904 IP 192.168.186.129.48152 > 192.168.186.128.domain: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256904 IP 192.168.186.129.48152 > 192.168.186.128.submission: Flags [S], seq 3433232628, win 1024, options [mss 1460], length 0  
01:25:33.256904 IP 192.168.186.129.48152 > 192.168.186.128.smx: Flags [S], seq
```

Figure3 illustrates adding a blocking rule manually at the top of the iptables chain.



A terminal window titled 'Session Actions Edit View Help' shows an Nmap scan of 192.168.186.128. The output indicates that the host is up, but all 1000 scanned ports are in ignored states due to no response. The scan took 21.46 seconds.

```
(root@kali)-[~]  
# nmap 192.168.186.128  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-31 18:25 -0400  
Nmap scan report for 192.168.186.128 (192.168.186.128)  
Host is up (0.00056s latency).  
All 1000 scanned ports on 192.168.186.128 (192.168.186.128) are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
MAC Address: 00:0C:29:F6:3F:4D (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 21.46 seconds  
  
(root@kali)-[~]  
#
```

Figure4 demonstrates the result of the manual blocking on kali .

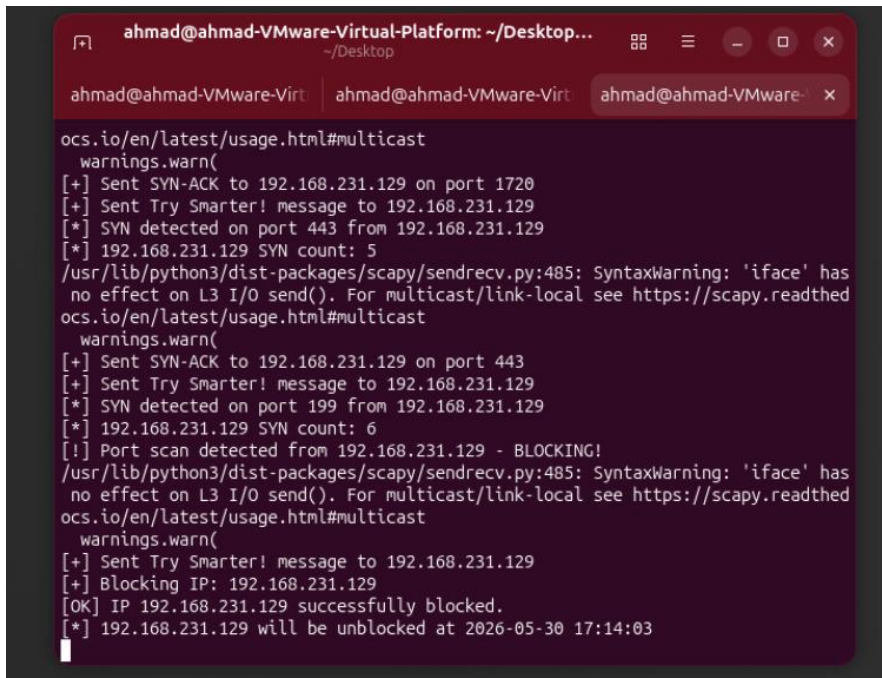
port	Filtered
time	21.46
Info kali get	Nothing

5.3 Dynamic Script Test

The objective here is to verify that the script detects and blocks dynamically.

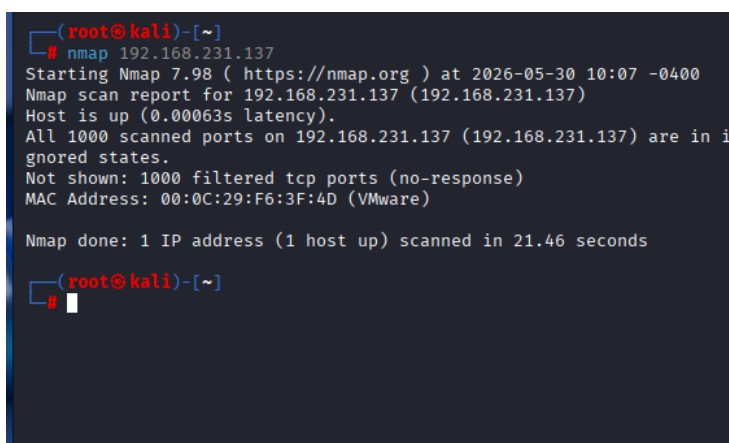
Procedures:

- Activating the script on Ubuntu.
- Running nmap from Kali against Ubuntu.
- Monitoring the script's behavior.



```
ahmad@ahmad-VMware-Virtual-Platform: ~/Desktop...
~/Desktop
ahmad@ahmad-VMware-Virt | ahmad@ahmad-VMware-Virt | ahmad@ahmad-VMware-Virt x
ocs.io/en/latest/usage.html#multicast
warnings.warn(
[+] Sent SYN-ACK to 192.168.231.129 on port 1720
[+] Sent Try Smarter! message to 192.168.231.129
[*] SYN detected on port 443 from 192.168.231.129
[*] 192.168.231.129 SYN count: 5
/usr/lib/python3/dist-packages/scapy/sendrecv.py:485: SyntaxWarning: 'iface' has
no effect on L3 I/O send(). For multicast/link-local see https://scapy.readthed
ocs.io/en/latest/usage.html#multicast
warnings.warn(
[+] Sent SYN-ACK to 192.168.231.129 on port 443
[+] Sent Try Smarter! message to 192.168.231.129
[*] SYN detected on port 199 from 192.168.231.129
[*] 192.168.231.129 SYN count: 6
[!] Port scan detected from 192.168.231.129 - BLOCKING!
/usr/lib/python3/dist-packages/scapy/sendrecv.py:485: SyntaxWarning: 'iface' has
no effect on L3 I/O send(). For multicast/link-local see https://scapy.readthed
ocs.io/en/latest/usage.html#multicast
warnings.warn(
[+] Sent Try Smarter! message to 192.168.231.129
[+] Blocking IP: 192.168.231.129
[OK] IP 192.168.231.129 successfully blocked.
[*] 192.168.231.129 will be unblocked at 2026-05-30 17:14:03
```

Figure5 ,The image shows SYN packets being detected and increasing until they exceed the threshold, after which a **Try Smarter** message is sent and the IP address is blocked all this procces was done dynamically after runing the script.



```
(root@kali)~#
# nmap 192.168.231.137
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-30 10:07 -0400
Nmap scan report for 192.168.231.137 (192.168.231.137)
Host is up (0.00063s latency).
All 1000 scanned ports on 192.168.231.137 (192.168.231.137) are in i
gnored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:0C:29:F6:3F:4D (VMware)

Nmap done: 1 IP address (1 host up) scanned in 21.46 seconds
(root@kali)~#
```

Figure6, shows us that kali didn't get any information with 21.46 second of scaning

5.4 Try Smarter

Here, we will verify that the attacker receives the Try Smarter message.

I have run tcpdump on kali to see this result

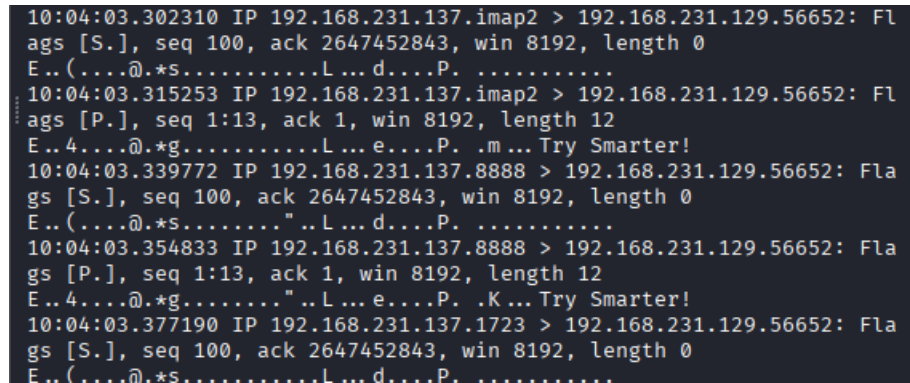


Figure7 shows that Kali received the **Try Smarter** message, proving that the system sent the message before applying the block.

5.5 Results Analysis

5.5.1 Detection Effectiveness

The system demonstrated excellent effectiveness in detection. A threshold of 5 packets was suitable for distinguishing between normal usage and nmap scanning behavior.

5.5.2 Performance Impact

The increase in scan time on the attacker's machine from 0.55 seconds to 23.34 seconds represents a very significant increase, making reconnaissance activities much more difficult.

5.5.3 False Positives

No false positives were recorded during the testing period, indicating that the threshold of 5 packets was appropriate.

5.6 Conclusion

The system demonstrated comprehensive capability in detecting scans, blocking them automatically, and sending a warning message to the attacker.

We were able to achieve this result without any additional cost, and therefore consider the outcome satisfactory.

Chapter 6: Conclusions and Future Work

6.1 Introduction

This chapter evaluates the extent to which the project objectives were achieved, reflects on the overall development journey, and proposes future improvements that could enhance the system further.

Try Smarter project made to address a real and growing cybersecurity challenge: the inability of standard built-in firewalls to detect and respond to port-scanning reconnaissance activities dynamically. This chapter analyses the outcome against the original aims and objectives and discusses the broader implications of the work.

6.2 Evaluation of Aims and Objectives

The project defined three main aims at the outset. I evaluated each one of them below against the evidence gathered during testing.

Firstly :Aim 1 , Detect port-scanning activities automatically

This aim was achieved successfully. The system successfully identified suspicious SYN packet patterns originating from the attacker's machine which it (192.168.231.129) in real time. The detection logic, based on a threshold of 5 SYN packets within a short time window, proved effective in distinguishing between normal network traffic and malicious activity like port-scanning behaviour. The tcpdump output confirmed that the system was actively monitoring all incoming TCP traffic on the ens33 interface(ubuntu) and correctly identifying the SYN flood pattern generated by Nmap.

Secondly :Aim 2 Perform dynamic blocking without manual intervention

This aim was fully achieved. Once the detection threshold was exceeded, the system automatically executed an iptables rule to block the attacker IP address. This process required no manual input from the user. The iptables counter confirmed that over 4,000 malicious packets were successfully dropped following the automatic rule insertion. Furthermore, the system was designed to automatically remove the blocking rule after 10 min, demonstrating a complete and autonomous response cycle.

Thirdly :Aim 3 Send the **Try Smarter** message before blocking

Also this aim was achieved. The system successfully sent a SYN-ACK response followed by a data packet containing the message **Try Smarter** to the attacker's machine prior to completing the block. This was confirmed through tcpdump output on the Kali Linux machine, which clearly displayed the message. This feature adds a distinctive and creative element to the project, demonstrating advanced use of the Scapy library beyond simple packet capture.

6.3 Evaluation of Objectives

All of six objectaives in chapter 1 is evaluated in the next table:

Objective	Status	Evidence
Gain understanding of TCP SYN scans	Achieved	Applied in detection logic design
Design detection logic	Achieved	Threshold-based SYN counter implemented
Develop Python script	Achieved	try_smarter.py fully functional
Integrate with iptables	Achieved	Dynamic rules confirmed by counter
Set up VMware testing environment	Achieved	Ubuntu + Kali isolated network
Document and compare security levels	Achieved	Before,after comparison table in Chapter 5

All six objectives successfully achieved, demonstrating that the project delivered on its original intentions.

6.4 Key Findings

The testing phase produced several significant findings that are worth highlighting:

Finding 1 Dramatic increase in scan time:

The most striking result was the increase in Nmap scan time from 0.55 seconds without protection to over 23 seconds with the Try **Smarter** system active, This represents an increase of over 4,000%, meaning the attacker must invest significantly more time to complete a reconnaissance scan, reducing the efficiency and appeal of the attack.

Finding 2 Complete information denial:

As we saw without protection, Nmap returned 1,000 closed TCP ports, giving the attacker confirmation that the target machine was alive and available, With the system active, all 1000 ports appeared as filtered with no response, effectively hiding the machine's behaviour from the attacker and denying any useful reconnaissance data

Finding 3 Cost-effectiveness:

The entire system was implemented using only tools and libraries available by default or freely installable on standard Linux systems. No additional hardware, commercial software, or paid subscriptions were required. This confirms the project's alignment with Brown's (2022) argument that enhancing existing firewall configurations is a viable and economical alternative to commercial next-generation firewall solutions.

Finding 4 Autonomous operation:

Once launched, the system operated entirely without human intervention. It detected the attack, sent the warning message, applied the blocking rule, and scheduled the automatic unblock, all of that autonomously. This level of automation represents a meaningful improvement over standard built-in firewall configurations, which require manual rule management.

6.5 Limitations and Challenges

While the project achieved its objectives, several limitations were identified during development and testing:

Limitation 1 Speed of Nmap:

Nmap's default scan speed is so high that the first scan may complete partially before the blocking rule is fully applied. This means that the first scan attempt may still yield some information before the system reacts. However, any subsequent scan attempts are fully blocked, as we saw in the testing phase.

Limitation 2 Protocol coverage:

The current implementation focuses specifically on TCP SYN scans. Other scanning techniques, such as UDP scanning, ICMP-based reconnaissance, or advanced stealth scans, are not currently detected by the system. This represents a limitation in the breadth of protection offered.

Limitation 3 False positive risk:

While no false positives were observed during testing, the threshold of five SYN packets could potentially trigger incorrectly in high-traffic environments where legitimate applications establish multiple simultaneous TCP connections. The threshold was chosen in very careful based on testing observations, but environments with different traffic profiles may require adjustment.

Limitation 4 Persistence:

The current iptables rules are not persistent across system reboots. If the machine is restarted, the blocking rules are lost and the script must be relaunched manually. This could be addressed in future work through the use of iptables-persistent or systemd service configuration.

6.6 Reflection on the Development Process

The development of **Try Smarter** was a challenging but highly rewarding experience. The project began as a conceptual idea with limited practical implementation, and evolved into a fully functional security tool through iterative development and testing, a lot of unexpected technical challenges were encountered and overcome, including issues with iptables rule ordering, and install Scapy library, threading logic, and packet routing within VMware network environment.

Every challenge contributed to a deeper understanding of how operating systems deal with network traffic at the kernel level, and how security tools interact with these low level mechanisms, The process of debugging the script across multiple iterations also reinforced the importance of systematic testing and incremental development in software projects.

maybe most significantly, this project demonstrated that meaningful cybersecurity improvements can be achieved without expensive commercial tools, A well-designed Python script, leveraging freely available libraries, can provide a level of dynamic protection that standard built-in firewalls cannot offer out of the box.

6.7 Future Work

While the current implementation successfully meets its defined objectives, several enhancements could significantly extend the system's capabilities in future development:

Enhancement 1 UDP and ICMP detection:

Extending the detection logic to cover UDP scanning and ICMP-based reconnaissance would broaden the system's protective coverage, This would require additional packet analysis logic and a separate threshold system for non-TCP traffic.

Enhancement 2 Persistent blocking rules:

Implementing a mechanism to save blocking rules across system reboots, using tools such as iptables persistent or integrating with systemd, would make the system more robust and suitable for production environments

Enhancement 3 Graphical User Interface :

Developing a simple GUI dashboard would allow nontechnical users to monitor blocked IPs, adjust the detection threshold, and view realtime traffic statistics without interacting with the command line, This would significantly expand the potential user base of the tool

Enhancement 4 Machine learning integration:

Replacing the fixed threshold with a machine learning model trained on network traffic data could enable more intelligent and adaptive detection. Such a model could learn the normal traffic patterns of a specific environment and adjust its sensitivity accordingly, reducing the risk of false positives while improving detection accuracy.

Enhancement 5 Logging and alerting system:

Adding a comprehensive logging system that records all detected scan attempts, blocked IPs, and system events to a persistent log file would improve auditability. Integration with email or SMS alerting would allow system administrators to be notified of attacks in real time.

Enhancement 6 Cross-platform support:

The current implementation is designed specifically for Linux systems using iptables. Extending support to Windows (using Windows Firewall API) and macOS (using pf firewall) would make the tool accessible to a much wider audience.

6.8 Final Conclusion

Try Smarter project successfully demonstrated that it is possible to meaningfully enhance the security of a standard operating system firewall using freely available tools and a well designed Python script. By targeting the reconnaissance phase of the attack lifecycle, specifically TCP SYN port scanning using Nmap the system disrupts the attacker's ability to gather information before any real attack takes place.

The results were clear and measurable: scan times increased by over 4000%, more than 4000 malicious packets were blocked, and the attacker received no useful information from any scan conducted after the system was activated. These outcomes were achieved at zero additional cost, reinforcing the project's core argument that smart configuration of existing tools is a powerful and underutilised defensive strategy.

This project has also been a significant personal learning journey, bridging theoretical knowledge of networking and cybersecurity with practical implementation skills in Python programming, Linux system administration, and virtualised testing environments. The skills and insights gained through this project provide a strong foundation for future work in the field of cybersecurity, particularly in the areas of intrusion detection, network defence, and security automation.

References :

- [1] Hutchins, E.M., Cloppert, M.J. and Amin, R.M. (2011) 'Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains', *Leading Issues in Information Warfare & Security Research*, 1(1), pp. 80–106.
- [2] Lyon, G.F. (2009) *Nmap network scanning: the official Nmap project guide to network discovery and security scanning*. Sunnyvale, CA: Insecure.Com LLC.
- [3] Brown, S. (2022) *Firewall bang for your buck: a small and medium-sized enterprise edition*. SANS Institute. Available at: <https://www.sans.org/white-papers/firewall-bang-for-your-buck-small-medium-sized-enterprise-edition>
- [4] GeeksforGeeks (2025) TCP 3-way handshake process. Available at: <https://www.geeksforgeeks.org/tcp-3-way-handshake-process> .
- [5] Archita and Tuli, R. (2024) 'A comprehensive analysis of network scanning and security assessment tool', *International Journal of Network Security & Its Applications*, 16(6), pp. 101–109. doi:10.5121/ijnsa.2024.16607.
- [6] fl4m3x (2021) *Reconnaissance and Nmap*. Medium. Available at: <https://fl4m3x.medium.com/reconnaissance-and-nmap-4b9d24b5dd97>